

Loyola University Chicago
Department of Computer Science

COMP 170: Introduction to Object-Oriented Programming
Practice Questions

October 1, 2026



What these questions are for. These practice questions walk you through the concepts we have covered in class so far, one topic at a time, so you can see what you should be reviewing and which ideas still need more work. They are grouped in six sections:

1. Errors, data types, operators, naming and variables (including `print`, strings and `input`).
2. Pseudocode and functions (parameters, `return`, annotations, docstrings, `main`).
3. `for` loops (`range`, nested loops, cumulative algorithms).
4. Conditional execution (`if/elif/else`, Boolean expressions).
5. Lists (creating, accessing, modifying, slicing and resizing lists).
6. Modules, the `math` module, Vim and terminal commands.

If a question feels hard, go back to the notes and code for that topic before moving on.

Question formats. The quiz will use the same question formats you see here: multiple-choice questions (sometimes “check all that apply”), short questions (matching, writing a value, a one- or two-sentence explanation), and code questions where you read, correct or complete code.

How to use them. Answer every question on paper first, without running anything. Then look at the answered version, type the code into a file, and run it with `python3` to test it.

Unless stated otherwise, assume all code is Python 3 and runs top to bottom. Each question is independent. Short tracing snippets may omit pseudocode, docstrings, type annotations or `main()` for brevity. Complete programs should follow the course requirements. The argument type in the `shout` question is intentionally unspecified because the function checks it at runtime.

Section 1 — Errors, data types, operators, naming & variables

1. **Types of errors.** Each version below should read a price and a quantity and print the total cost. The user types 2.5, then 4, so the correct output is **Total: 10.0**. Link each version to its result by writing the result number (1, 2 or 3) next to it. Each result is used exactly once.

Version A

```
price = float(input("Price?
"))
qty = int(input("Qty? "))
print("Total: " + price *
      qty)
```

Version B

```
price = float(input("Price?
"))
qty = int(input("Qty? "))
print("Total:", price + qty
      )
```

Version C

```
price = float(input("Price?
"))
qty = int(input("Qty? "))
print("Total:" price * qty)
```

Results:

1. Python reports a **SyntaxError**, and nothing runs.
2. The program stops while running with a **TypeError** (runtime error).
3. The program runs, but prints a wrong answer (logic error).

Version A → _____ Version B → _____ Version C → _____

Solution: A → 2: "Total: " + 10.0 tries to join a string and a float (**TypeError**). B → 3: it adds instead of multiplying and prints **Total: 6.5**. C → 1: the comma between "Total:" and `price * qty` is missing (**SyntaxError**).

2. What does `print(type(7 / 7), type(7 // 7), type("7"))` display?

- ☐ <class 'int'> <class 'int'> <class 'int'>
- ☒ <class 'float'> <class 'int'> <class 'str'>
- ☐ <class 'int'> <class 'float'> <class 'str'>
- ☐ <class 'float'> <class 'float'> <class 'int'>

Solution: / always gives a float (1.0); // of two ints gives an int (1); anything in quotes is a string.

3. **Precedence.** What does `print(2 + 3 * 4 ** 2 // 8)` display?

- ☒ 8
- ☐ 10
- ☐ 20
- ☐ 6

Solution: ** first: $4^2 = 16$. Then * and // left to right: $3 \times 16 = 48$, $48 // 8 = 6$. Finally $2 + 6 = 8$.

4. **Operators and types.** After this code runs, what are the values of `x`, `y` and `z`? Write each value exactly as Python stores it (use quotes for strings).

```
1 x = "ab" * 3
2 y = "5" + "5" * 2
3 z = 17 % 5 * 2
```

`x` = _____ `y` = _____ `z` = _____

Solution: `x` = "ababab"; `y` = "555" (* runs before +, so "5" * 2 is "55", then it is joined to "5");
`z` = 4 (17%5 = 2, then 2 × 2).

5. **Naming.** Which of these are **not** valid Python variable names? Check **all** that apply.

- ☒ `2nd_place`
☒ `first-name`
☐ `_count`
☒ `class`
☐ `total2`

Solution: A name cannot start with a digit, cannot contain a hyphen (Python reads it as minus), and cannot be a keyword such as `class`. Reminder: our convention is **snake_case** for variables and **UPPER_CASE** for constants.

6. **Variables.** What is printed?

```
1 a = 3
2 b = a
3 a = a * 2
4 b = b + a
5 print(a, b)
```

- ☐ 6 6
☐ 6 12
☒ 6 9
☐ 3 9

Solution: `b` refers to the value 3; assigning a new value to `a` does not change `b`. Then `a` becomes 6, and `b` becomes 3 + 6 = 9.

7. **Strings and escape sequences.** Which statement prints exactly `It's a "test"`?

- ☐ `print("It's a "test")`
- ☐ `print('It's a "test"')`
- ☒ `print("It's a \"test\"")`
- ☐ `print('It's a \"test\"')`

Solution: Inside double quotes, the apostrophe is fine but each inner `"` must be escaped as `\`. In (a) the string ends early; in (b) and (d) the apostrophe ends the single-quoted string.

8. The user types `3.9` at the prompt. What happens?

```
1 n = int(input("n? "))
2 print(n + 1)
```

- ☐ It prints 4
- ☐ It prints 4.9
- ☐ It prints 5
- ☒ The program stops with a `ValueError`

Solution: `int()` cannot convert the string `"3.9"`; it only accepts text that looks like a whole number. Use `float()` for decimals.

9. What does `print(7 / 2 * 2, 7 // 2 * 2)` display?

- ☐ `7 7`
- ☒ `7.0 6`
- ☐ `7.0 7.0`
- ☐ `6 6`

Solution: $7/2 = 3.5$, times 2 is 7.0 (a float). $7//2 = 3$, times 2 is 6.

10. Which statements print exactly `Score: 10`? Check **all** that apply.

- ☐ `print("Score: " + 10)`
- ☒ `print("Score:", 10)`
- ☒ `print("Score: " + str(10))`
- ☐ `print("Score:" + "10")`

Solution: (a) raises a `TypeError`: `+` cannot join a string and an int. (b) commas print the values separated by one space. (c) `str()` turns 10 into text first. (d) prints `Score:10`, with no space.

Section 2 — Pseudocode & functions

11. What is printed?

```
1 def triple(n: int) -> int:
2     return n * 3
3     print("done")
4
5 print(triple(2))
```

☒ $\sqrt{6}$ ☐ done then 6☐ 6 then done☐ None

Solution: `return` ends the function immediately, so the `print("done")` line never runs.

12. **Local variables.** What is printed?

```
1 def set_total() -> None:
2     total = 10
3
4 total = 5
5 set_total()
6 print(total)
```

☒ $\sqrt{5}$ ☐ 10☐ 15☐ A NameError

Solution: The `total` inside `set_total` is a separate, local variable. It disappears when the function ends; the outer `total` is still 5.

13. What is printed?

```
1 def main() -> None:
2     print("B")
3
4 print("A")
5 main()
6 print("C")
```

☐ B A C

☒ A B C

☐ A C B

☐ A C

Solution: Defining `main` runs nothing. The program prints A, then the call `main()` prints B, then C.

14. Which statement(s) about this function are **false**? Check **all** that apply.

```

1 def average(a: float, b: float) -> float:
2     """Return the average of a and b."""
3     return (a + b) / 2

```

- ☐ average(4, 6) returns 5.0.
- ☒ The annotation `-> float` makes Python convert the result to a float.
- ☐ Calling `average(4)` raises an error, because one argument is missing.
- ☒ The docstring is printed every time `average` is called.
- ☐ `a` and `b` are parameters.

Solution: Annotations are notes for readers; Python does not enforce or convert anything. A docstring documents the function; it is never printed by a call.

15. A function receives a person's name (text) and age (a whole number) and gives back a greeting sentence. Which header is correct?

- ☐ `def describe(name: str, age: int) -> None:`
- ☒ `def describe(name: str, age: int) -> str:`
- ☐ `def describe(name: string, age: integer) -> str:`
- ☐ `def describe(name = str, age = int) -> str:`

Solution: The types are `str` and `int`, written after a colon; the function gives back a string, so `-> str`. (d) sets default values instead of annotations.

16. **Fill in the blanks.** Complete the program so that it asks for a temperature in Celsius and prints it in Fahrenheit ($F = C \times 9/5 + 32$).

```

1 def celsius_to_fahrenheit(c: float) -> float:
2     """Return the temperature c converted to Fahrenheit."""
3     _____(1)_____
4
5 def main() -> None:
6     """Ask for a Celsius temperature and print it in Fahrenheit."""
7     c = float(input("Celsius? "))
8     f = _____(2)_____
9     print(c, "C is", f, "F")
10
11 main()

```

Solution: (1) `return c * 9 / 5 + 32` (it must *return*, not `print`). (2) `celsius_to_fahrenheit(c)`.

17. **Pseudocode, docstrings and annotations.** The program below works, but it has no pseudocode, no docstrings and no type annotations.

```
1 def carpet_cost(length, width, price):
2     area = length * width
3     return area * price
4
5
6 def main():
7     length = float(input("Length (ft)? "))
8     width = float(input("Width (ft)? "))
9     cost = carpet_cost(length, width, 4.5)
10    print("Carpet cost: $" + str(round(cost, 2)))
11
12 main()
```

- (a) Write the pseudocode for the whole program (both functions).

Solution:

carpet_cost: receives the length, the width and the price per square foot
compute the area as length x width
give back the area x price

main:

ask the user for the length and the width (decimal numbers)
call carpet_cost with the length, the width and 4.5
print "Carpet cost: \$" followed by the cost rounded to 2 decimals

- (b) Rewrite both function headers with type annotations, and add a docstring to each function.

Solution:

```
1 def carpet_cost(length: float, width: float, price: float) -> float:
2     """Return the cost of carpeting a length x width room at price per sq ft"""
3
4 def main() -> None:
5     """Ask for the room size and print the carpet cost."""
```

18. **Correct the code.** This program should print `Perimeter: 14` for a 3 by 4 rectangle. It contains **three** bugs. Identify each bug and write the corrected program.

```
1 def rectangle_perimeter(width: float, height: float) -> float:
2     """Return the perimeter of a rectangle."""
3     perimeter = 2 * width + height
4     print(perimeter)
5
6 def main() -> None:
7     """Print the perimeter of a 3 by 4 rectangle."""
8     p = rectangle_perimeter(3, 4)
9     print("Perimeter:", p)
```

Solution:

1. `2 * width + height` computes 10, not 14 (precedence). Use `2 * (width + height)`.
2. The function prints instead of returning, so `p` is `None`. Use `return perimeter`.
3. `main` is defined but never called. Add `main()` at the end.

19. **Default parameter values.** What is printed?

```
1 def price_with_tax(price: float, rate: float = 0.08) -> float:
2     """Return price plus tax at the given rate, rounded to cents."""
3     return round(price * (1 + rate), 2)
4
5 print(price_with_tax(100), price_with_tax(100, 0.1))
```

- ☐ 108.0 108.0
- ☒ 108.0 110.0
- ☐ An error, because the first call is missing an argument
- ☐ 100 110.0

Solution: When the caller leaves out `rate`, the default 0.08 is used; the second call passes 0.1 instead.

Section 3 — for loops

20. What is printed?

```
1 for i in range(2, 11, 3):  
2     print(i, end=" ")
```

- ☐ 2 5 8 11
- ☒ 2 5 8
- ☐ 3 6 9
- ☐ 2 4 6 8 10

Solution: Start at 2 and add 3 each time; stop *before* 11.

21. Which loop prints 5 4 3 2 1?

- ☒ for i in range(5, 0, -1): print(i, end=" ")
- ☐ for i in range(5, 1, -1): print(i, end=" ")
- ☐ for i in range(5, 0): print(i, end=" ")
- ☐ for i in range(1, 6, -1): print(i, end=" ")

Solution: Counting down needs a negative step, and the stop value (0) is not included. (b) stops at 2; (c) and (d) print nothing.

22. **Cumulative algorithms.** What is printed?

```
1 total = 1  
2 for k in range(1, 5):  
3     total *= k  
4     print(total)
```

- ☒ 24
- ☐ 10
- ☐ 120
- ☐ 0

Solution: $1 \times 1 \times 2 \times 3 \times 4 = 24$. `range(1, 5)` stops before 5. (If `total` started at 0, every product would be 0.)

23. After this code runs, what are the values of `total` and `i`?

```
1 total = 0
2 for i in range(10, 0, -2):
3     total += i
```

`total` = _____ `i` = _____

Solution: `total` = 30 ($10 + 8 + 6 + 4 + 2$) and `i` = 2, the last value the loop used.

24. What does this code print?

```
1 for _ in range(3):
2     print("hi", end="-")
```

- ☐ hi hi hi
- ☒ hi-hi-hi-
- ☐ hi-hi-hi
- ☐ hi on three separate lines

Solution: `_` means “we do not need the counter”. `end="-"` replaces the newline with a dash, including after the last `hi`.

25. Nested loops. Consider:

```
1 for row in range(3):
2     for col in range(4):
3         print("#", end=" ")
4     print()
```

(a) What does it print?

- ☒ 3 lines of 4 #
- ☐ 4 lines of 3 #
- ☐ 12 # on one line
- ☐ 3 lines of 3 #

(b) Rewrite the inner loop header so that the program prints this triangle instead:

```
#
##
###
```

```
for col in _____:
```

Solution: (i) The outer loop runs 3 times; each time the inner loop prints 4 #, then `print()` ends the line. (ii) `range(row + 1)`: row 0 prints 1, row 1 prints 2, row 2 prints 3.

26. **Fill in the blanks.** Complete the loop so that, with `SIZE = 4`, it prints this right-aligned triangle. Use string multiplication.

```
*
**
***
****
```

```
1 SIZE = 4
2 for i in _____(1)_____:
3     print(" " * _____(2)_____ + "*" * i)
```

Solution: (1) `range(1, SIZE + 1)`; (2) `SIZE - i`. Line i has $SIZE - i$ spaces and i stars. Using `SIZE` (not 4) makes the figure scale.

27. **Correct the code.** `sum_of_squares(3)` should return $1 + 4 + 9 = 14$. The code contains **three** bugs. Identify each bug and write the corrected function.

```
1 def sum_of_squares(n: int) -> int:
2     """Return 1*1 + 2*2 + ... + n*n."""
3     for i in range(n):
4         total = 0
5         total = i * i
6     return total
```

Solution:

1. `total = 0` is inside the loop, so the total resets every pass. Move it before the loop.
2. `range(n)` gives $0, \dots, n - 1$. Use `range(1, n + 1)`.
3. `total = i * i` replaces the total instead of adding to it. Use `total += i * i`.

```
1 def sum_of_squares(n: int) -> int:
2     """Return 1*1 + 2*2 + ... + n*n."""
3     total = 0
4     for i in range(1, n + 1):
5         total += i * i
6     return total
```

28. **Constants, modify-and-assign and string multiplication.** After this code runs, what are the values of `x` and `line`?

```
1 SIZE = 5
2 x = 10
3 x -= 3
4 x *= 2
5 x //= 4
6 x %= 3
7 line = "+" + "-" * SIZE + "+"
```

`x` = _____ `line` = _____

Solution: `x` = 0: $10 - 3 = 7$, $7 \times 2 = 14$, $14 // 4 = 3$, $3 \% 3 = 0$. `line` = "+-----+" (five dashes). Changing `SIZE` in one place resizes the line, which is why we use a named constant.

Section 4 — Conditional execution

29. What is printed?

```
1 age = 20
2 has_id = False
3 if age >= 18 or has_id and age < 18:
4     print("Enter")
5 else:
6     print("Stop")
```

- ☒ Enter
- ☐ Stop
- ☐ Enter then Stop
- ☐ An error is raised

Solution: and has higher precedence than or, so the condition groups as `age >= 18 or (has_id and age < 18)`. The first comparison is true, so the whole condition is true and the program prints Enter.

30. With `x = 4` and `y = 9`, which expressions are True? Check **all** that apply.

- ☒ `x < y and y < 10`
- ☐ `not x == 4`
- ☒ `x > 5 or y % 3 == 0`
- ☐ `x != 4 or y > 10`

Solution: (a) both parts are true. (b) `x == 4` is true, so `not` makes it false. (c) `9 % 3 == 0` is true, and `or` needs only one. (d) both parts are false.

31. What is printed?

```
1 score = 95
2 if score >= 90:
3     print("A")
4 if score >= 80:
5     print("B")
6 else:
7     print("C")
```

- ☐ A

- ☒ A then B
- ☐ A then C
- ☐ B

Solution: These are two separate `if` statements, so both tests run. This is the “misuse of `if`” from class: a grade needs `if/elif/else`.

32. **Choose the structure.** For each scenario, write the letter of the structure that fits best. Not every structure is used.

A. if alone **B.** if / else **C.** if / elif / else **D.** if / elif (no else) **E.** separate if statements (if / if / if)

1. Print "Even" or "Odd" for a whole number. _____
2. Classify a yearly income as low, middle or high. _____
3. Print a warning only when the battery is below 10%. _____
4. Print "Gold", "Silver" or "Bronze" for places 1, 2 and 3; print nothing for any other place. _____
5. For a password, print "Too short", "No digit" and/or "No capital": every problem that applies. _____

Solution: 1 → B, 2 → C, 3 → A, 4 → D, 5 → E.

33. What value does `category` hold when `temp = 72`? And when `temp = 50`? Write each value as Python stores it.

```

1 if temp < 50:
2     category = "cold"
3 elif temp < 70:
4     category = "mild"
5 elif temp < 85:
6     category = "warm"
7 else:
8     category = "hot"

```

`temp = 72:` _____ `temp = 50:` _____

Solution: "warm" and "mild". Python stops at the first true test; 50 is not below 50, so the second test decides.

34. Which condition is **True** exactly when `x` is between 1 and 10, both included?

- ☒ `x >= 1 and x <= 10`
- ☐ `x >= 1 or x <= 10`
- ☐ `x >= 1 and <= 10`
- ☐ `not (x < 1 and x > 10)`

Solution: (b) is true for every number; (c) is a **SyntaxError** (each side of **and** needs a full comparison); (d) is always true, because no number is both below 1 and above 10.

35. **Fill in the blank.** Complete the condition using `type()` so that the function returns `word` followed by `!!!` when `word` is a string, and returns the empty string `""` otherwise.

```

1 def shout(word) -> str:
2     """Return word followed by '!!!' if it is a str; otherwise return '!'."""
3     result = ""
4     if -----:
5         result = word + "!!!"
6     return result

```

Expected: `shout("hey") → "hey!!!"`, `shout(5) → ""`.

Solution:

```

1     if type(word) == str:

```

The condition checks whether the argument is a string. If so, the function updates `result`; otherwise it remains the empty string. The function has one final `return` statement.

36. **Correct the code.** `ticket_price` should return 0 for ages under 5, 8 for ages 5 to 12, 12 for ages 13 to 64, and 6 for ages 65 and over. The code contains **two** bugs. Identify each bug and write the corrected function.

```

1 def ticket_price(age: int) -> int:
2     """Return the ticket price in dollars for a visitor of the given age."""
3     if age < 5:
4         price = 0
5     if age <= 12:
6         price = 8
7     elif age > 65:
8         price = 6
9     else:
10        price = 12
11    return price

```

Solution:

1. The second `if` starts a new, independent test, so a 3-year-old is charged 8. Change it to `elif`.
2. `age > 65` misses age 65, which is then charged 12. Use `age >= 65`.

```

1 def ticket_price(age: int) -> int:
2     """Return the ticket price in dollars for a visitor of the given age."""
3     if age < 5:
4         price = 0
5     elif age <= 12:
6         price = 8

```

```
7 elif age >= 65:  
8     price = 6  
9 else:  
10     price = 12  
11 return price
```

37. **Bounding with min and max.** Which single statement keeps `score` between 0 and 100 (negative scores become 0, scores above 100 become 100, others stay the same)?

- ☐ `score = min(0, max(100, score))`
- ☒ `score = max(0, min(100, score))`
- ☐ `score = max(0, score) + min(100, score)`
- ☐ `score = min(0, score)`

Solution: `min(100, score)` caps the score at 100, and `max(0, ...)` never lets it go below 0. (a) always gives 0.

Section 5 — Lists

38. What is printed?

```
1 letters = ["a", "b", "c", "d", "e"]
2 print(letters[-3:], letters[1])
```

- ☒ ['c', 'd', 'e'] b
- ☐ ['c', 'd'] a
- ☐ ['a', 'b', 'c'] b
- ☐ ['d', 'e'] b

Solution: [-3:] is the last three elements; index 1 is the *second* element.

39. What is printed?

```
1 names = ["Ana", "Ben"]
2 names.insert(1, "Cy")
3 names.append("Dee")
4 names.remove("Ana")
5 last = names.pop()
6 print(names, last)
```

- ☒ ['Cy', 'Ben'] Dee
- ☐ ['Ben', 'Cy'] Dee
- ☐ ['Cy', 'Ben', 'Dee'] Dee
- ☐ ['Ana', 'Cy'] Ben

Solution: ['Ana', 'Cy', 'Ben'] → ['Ana', 'Cy', 'Ben', 'Dee'] → ['Cy', 'Ben', 'Dee'] → pop() removes and gives back 'Dee'.

40. After this code runs, what are the values of a, b and c?

```
1 a = [0] * 3
2 b = [1, 2] + [3, 4]
3 c = len(b)
```

a = _____ b = _____ c = _____

Solution: a = [0, 0, 0], b = [1, 2, 3, 4], c = 4. Like strings, * repeats a list and + joins two lists.

41. What is printed?

```
1 x = [1, 2, 3]
2 y = x
3 y[0] = 99
4 print(x)
```

- ☐ [1, 2, 3]
- ☒ [99, 2, 3]
- ☐ [99]
- ☐ An error is raised

Solution: `y = x` gives the *same* list a second name; it is not a copy. Use `y = x[:]` for a separate copy.

42. What happens when this runs?

```
1 nums = [3, 6, 9]
2 print(nums[3])
```

- ☐ It prints 9
- ☐ It prints 3
- ☒ The program stops with an `IndexError`
- ☐ It prints None

Solution: A list of length 3 has indexes 0, 1, 2 (or -3 to -1). Index 3 is out of range.

43. **Fill in the blanks.** Complete the function so that it asks for `n` scores and returns them in a list.

```

1 def read_scores(n: int) -> list[float]:
2     """Ask for n scores and return them in a list."""
3     scores = _____(1)_____
4     for i in range(n):
5         _____(2)_____
6     return scores

```

Solution: (1) `[]` (an empty list); (2) `scores.append(float(input("Score " + str(i + 1) + ": ")))`.

44. **Correct the code.** `count_above([3, 7, 5, 9], 5)` should return 2. The code contains **three** bugs. Identify each bug and write the corrected function.

```

1 def count_above(values: list[int], limit: int) -> int:
2     """Return how many numbers in values are greater than limit."""
3     for i in range(len(values) - 1):
4         count = 0
5         if values[i] >= limit:
6             count += 1
7     return count

```

Solution:

1. `range(len(values) - 1)` skips the last element. Use `range(len(values))` (or `for v in values`).
2. `count = 0` is inside the loop, so it resets every pass. Move it before the loop.
3. `>=` also counts values equal to the limit. Use `>`.

```

1 def count_above(values: list[int], limit: int) -> int:
2     """Return how many numbers in values are greater than limit."""
3     count = 0
4     for i in range(len(values)):
5         if values[i] > limit:
6             count += 1
7     return count

```

Section 6 — Modules, Vim & Python commands

45. Which program prints 3.0?

- ☐ `import math then print(sqrt(9))`
- ☐ `from math import sqrt then print(math.sqrt(9))`
- ☒ `import math as m then print(m.sqrt(9))`
- ☐ `import math.sqrt then print(sqrt(9))`

Solution: `import math as m` names the module `m`. (a) and (b) raise `NameError`: each form only creates the names it imports. (d) is an error because `math.sqrt` is not a module.

46. Assuming `import math`, write the value of each expression.

`math.ceil(4.2)` = _____ `math.floor(-2.5)` = _____ `round(3.5)` = _____
`math.sqrt(49)` = _____

Solution: 5, -3 (floor goes down, even for negatives), 4 (ties go to the even number), 7.0 (`sqrt` always returns a float).

47. Which expression is True? (Assume `import math`.)

- ☐ `0.1 + 0.2 == 0.3`
- ☒ `math.isclose(0.1 + 0.2, 0.3)`
- ☐ `round(0.1 + 0.2) == 0.3`
- ☐ `int(0.3) == 0.3`

Solution: 0.1 and 0.2 cannot be stored exactly in binary, so their sum is 0.30000000000000004. Compare floats with `math.isclose`, not `==`.

48. A file `tools.py` contains only the line `print(__name__)`.

(a) What is printed when you run `python3 tools.py`?

- ☐ `tools`
- ☒ `__main__`
- ☐ `tools.py`
- ☐ Nothing

(b) What is printed when another file, `app.py`, contains only `import tools` and you run `python3 app.py`?

- ☒ `tools`
- ☐ `__main__`
- ☐ `app`
- ☐ Nothing, because importing a file does not run it

Solution: Python sets `__name__` to `"__main__"` only in the file you run. In an imported file it is the module name. And `import` does run the whole file once.

49. A student saves their own file as `math.py` in their project folder. In another file in the same folder they write `import math` and `print(math.sqrt(16))`, and get `AttributeError: module 'math' has no attribute 'sqrt'`. Explain in one or two sentences why this happens and how to fix it.

Solution: Python searches the program's own folder first, so `import math` loads the student's `math.py` instead of the real `math` module. Rename the file (for example `my_math.py`).

50. **Correct the code.** Both files are in the same folder. Running `python3 main.py` should print only the area of a circle of radius 2 (12.566...). The code in `circles.py` contains **two** problems. Identify each and write the corrected `circles.py`.

```
1 # circles.py
2 import math
3
4 def circle_area(r: float) -> float:
5     """Return the area of a circle of radius r."""
6     return pi * r ** 2
7
8 print(circle_area(1))    # quick test
```

```
1 # main.py
2 from circles import circle_area
3 print(circle_area(2))
```

Solution:

1. `pi` is not defined: with `import math`, write `math.pi` (`NameError` otherwise).
2. The quick test runs on import, so `main.py` would also print the area of radius 1. Put it under `if __name__ == "__main__":`.

```
1 # circles.py
2 import math
3
4 def circle_area(r: float) -> float:
5     """Return the area of a circle of radius r."""
6     return math.pi * r ** 2
7
8 if __name__ == "__main__":
9     print(circle_area(1))    # quick test
```

51. **Vim and terminal commands.** Link each command to what it does by writing the letter next to it. Each letter is used exactly once.

<code>pwd</code> _____	A. Leave Vim without saving
<code>ls</code> _____	B. Enter Insert mode in Vim
<code>cd comp170</code> _____	C. Show which folder you are in
<code>i</code> _____	D. Save the file and leave Vim
<code>Esc</code> _____	E. List the files in the current folder
<code>:wq</code> _____	F. Move into the folder <code>comp170</code>
<code>:q!</code> _____	G. Go back to Normal mode in Vim

Solution: `pwd` → C, `ls` → E, `cd comp170` → F, `i` → B, `Esc` → G, `:wq` → D, `:q!` → A.

52. You are in your home folder. Which sequence of terminal commands creates a folder named `comp170`, moves into it, and then shows where you are?

- ☐ `cd comp170, mkdir comp170, pwd`
- ☒ `mkdir comp170, cd comp170, pwd`
- ☐ `mkdir comp170, ls comp170, pwd`
- ☐ `vim comp170, cd comp170, ls`

Solution: The folder must exist before `cd` can move into it; `pwd` prints the current location. `ls` only lists files, and `vim comp170` opens a file named `comp170`.

53. **Importing your own code.** `helpers.py` defines a working function named `average` with the signature `def average(a: float, b: float) -> float:`. It is in the same folder as `main.py`, which starts with `import helpers`. Which line in `main.py` correctly calls the function?

- ☐ `print(average(8, 9))`
- ☒ `print(helpers.average(8, 9))`
- ☐ `print(helpers.py.average(8, 9))`
- ☐ `print(import helpers.average(8, 9))`

Solution: `import helpers` creates one name, `helpers`; the dot reaches inside the module for `average`.

54. Which of these file names can be imported as a module with `import name` (without `.py`)? Check **all** that apply.

- ☒ `my_helpers.py`
- ☐ `my-helpers.py`
- ☐ `my helpers.py`
- ☒ `helpers2.py`

Solution: A module name must be a valid Python name: no hyphens and no spaces.